

Introdução às APIs REST

O que é uma API REST?

Uma **API REST** é uma forma organizada e padronizada de expor funcionalidades e dados de um sistema utilizando o **protocolo HTTP**.

Ela permite que diferentes aplicações — como um **frontend web**, um **app mobile** ou até outro **backend** — se comuniquem entre si trocando informações sobre **recursos** (usuários, produtos, pedidos, etc.).

A grande vantagem desse modelo é a **facilidade de integração**, já que ele segue padrões amplamente utilizados na web moderna.

Conceito e Base Teórica

O que é uma API?

API significa **Application Programming Interface** (Interface de Programação de Aplicações).

Na prática, é um **conjunto de regras e contratos** que define como um software pode utilizar funcionalidades de outro, sem precisar conhecer seus detalhes internos.

O que é REST?

REST (Representational State Transfer) é um **estilo de arquitetura**, criado para guiar a comunicação entre sistemas distribuídos.

Ele define princípios como:

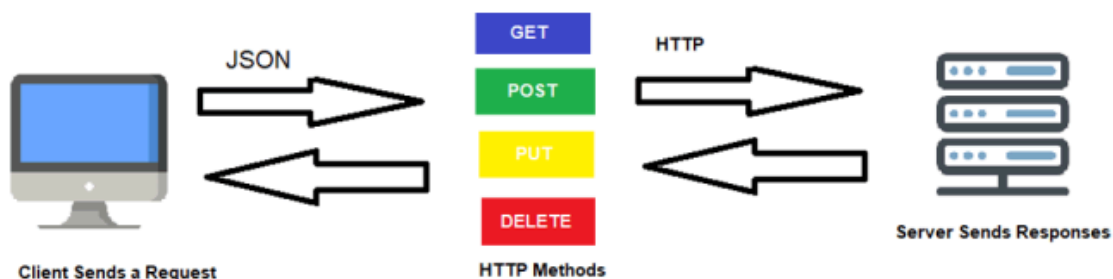
- Separação clara entre cliente e servidor
- Comunicação sem estado (stateless)
- Uso padronizado do protocolo HTTP

- Representação de recursos (normalmente em JSON)

👉 Uma **API RESTful** é, portanto, uma API que segue esses princípios.

🧱 Arquitetura Cliente–Servidor

No modelo REST, existe uma divisão clara de responsabilidades:



- **Cliente:** responsável pela interface e experiência do usuário
 - **Servidor:** concentra regras de negócio, autenticação e acesso a dados
-

⚙️ Princípios Importantes do REST

- ♦ **Stateless (Sem estado)**

Cada requisição é **independente**.

O servidor **não armazena informações de sessão** entre requisições.

➡ Isso significa que dados como autenticação (ex: JWT) devem ser enviados **em todas as requisições**.

- Requisição 1 → Token JWT → OK
- Requisição 2 → Token JWT → OK

◆ Cache

Sempre que possível, respostas podem ser **cacheadas**, melhorando desempenho e reduzindo carga no servidor.

Recursos, URLs e Métodos HTTP

Em uma API REST, tudo gira em torno de **recursos**.

Exemplos de recursos:

- `/users`
- `/products`
- `/orders`

Cada recurso é identificado por uma **URL (URI)**:

- `/users` → lista de usuários
- `/users/10` → usuário específico

Métodos HTTP

Os métodos HTTP indicam **a ação que será realizada** sobre um recurso:

Método	Descrição
GET	Recupera dados
POST	Cria um novo recurso
PUT	Atualiza um recurso por completo
PATCH	Atualiza parcialmente
DELETE	Remove um recurso

Exemplos práticos

- GET /users → Lista usuários
- POST /users → Cria usuário
- GET /users/10 → Busca usuário específico
- PUT /users/10 → Atualiza usuário
- DELETE /users/10 → Remove usuário

 Sugestão de imagem: tabela visual de métodos HTTP x CRUD

Códigos de Status HTTP

Os **status codes** informam ao cliente o resultado da requisição.

Faixas de códigos

- **2xx** → Sucesso
- **4xx** → Erro do cliente
- **5xx** → Erro do servidor

Códigos mais usados em APIs REST

Código	Significado
--------	-------------

200 OK	Sucesso
201 Created	Recurso criado
204 No Content	Sucesso sem retorno
400 Bad Request	Erro nos dados enviados
401 Unauthorized	Não autenticado
403 Forbidden	Sem permissão
404 Not Found	Recurso não encontrado
500 Internal Server Error	Erro interno

Componentes de uma Requisição REST

Uma chamada à API possui quatro partes principais:

- `POST /users`
- `Headers:`
 - `Content-Type: application/json`
 - `Authorization: Bearer <token>`
 -
- `Body:`
 - `{`
 - `"name": "João",`
 - `"email": "joao@email.com"`
 - `}`

♦ URL

Define **qual recurso** será acessado.

♦ Método HTTP

Define **qual ação** será realizada.

◆ Headers

Contêm metadados, como:

- Tipo de conteúdo (`application/json`)
- Autenticação (JWT)
- Cache

◆ Body

Usado em POST, PUT e PATCH, geralmente em **JSON**, por ser leve e legível.

Teoria Aplicada na Prática

Na prática, um backend expõe rotas como:

- `/api/users`
- `/api/orders`
- `/api/products`

O cliente (frontend ou outro serviço) consome essas rotas via HTTP para realizar operações de **CRUD**.

Ferramentas de teste como **Postman**, **Insomnia** ou **Thunder Client** ajudam a:

- Enviar requisições
- Analisar respostas
- Ver status codes e headers

 *Sugestão de imagem:* print de uma requisição no Postman

Boas Práticas em APIs REST

- URLs claras e padronizadas
 - Uso correto dos métodos HTTP
 - Respostas consistentes
 - Validação de dados no servidor
 - Mensagens de erro claras
 - Documentação da API (README ou Swagger)
-